

Securing IoT Using Lightweight TCN for Edge Deployment on Raspberry Pi 4

MIRZA AKHI^{ID} (Member, IEEE), CIARÁN EISING^{ID} (Senior Member, IEEE),
AND LUBNA LUXMI DHIRANI^{ID} (Senior Member, IEEE)

Department of Electronic and Computer Engineering, University of Limerick, Limerick, V94 T9PX Ireland

CORRESPONDING AUTHOR: M. AKHI (e-mail: Mirza.Akhi@ul.ie)

This work was supported by the Taighde Éireann—Research Ireland under Grant 18/CRT/6049.

ABSTRACT The Internet of Things (IoT) is a network of tiny sensing devices that facilitate precision monitoring, automation, and intelligent decision-making in a connected digital environment. While IoT enables high connectivity, it also presents various issues relating to compliance, operational resilience, coexistence, and cybersecurity. These factors significantly affect the robustness, reliability, and resilience of IoT devices. To mitigate emerging cybersecurity issues, an efficient and lightweight security solution is essential for resource-constrained edge devices. This research addresses security challenges by leveraging Healthcare-IoT (H-IoT) Distributed Denial-of-Service (DDoS) datasets to deploy a lightweight TCN model on a standalone IoT device, the Raspberry Pi 4. It enables efficient edge deployment for detecting critical cyber threats, particularly DDoS attacks. The TCN model is converted into TensorFlow Lite (TFLite) format and optimized through quantization, reducing model size and computational overhead. The deployment achieves 99.95% accuracy with an average inference latency of 0.19 ms on the MQTT-based dataset, and 99.94% accuracy with 0.27 ms on the UDP-based dataset. The average power consumption on the Raspberry Pi 4, measured using a physical USB power meter, is 4.22 W and 4.64 W for the two datasets. These results demonstrate the feasibility of high-accuracy, resource-efficient DDoS attack detection on low-power edge devices for securing IoT, Industrial IoT (IIoT), and H-IoT systems in real-world environments.

INDEX TERMS Cybersecurity, DDoS attack detection, edge computing, H-IoT, IIoT, IoT, TCN.

I. INTRODUCTION

THE ADVANCEMENT of a secure and sustainable digital ecosystem is based on the protection of infrastructures enabled by the Industrial Internet of Things (IIoT) [1], [2]. The Internet of Things (IoT) encompasses physical devices equipped with sensors and software that exchange data over networks [2], [3]. These devices are often domain-specific, such as smart home appliances, wearables in Healthcare-IoT (H-IoT), or industrial sensors in IIoT, yet all fall under the broader IoT paradigm of Internet-enabled, low-power communication. H-IoT devices support real-time health monitoring and enable precision and personalized care by continuously measuring vital signs such as body temperature, oxygen saturation, heart rate, and blood pressure [1], [3], [4]. Smart sensors and real-time tracking further enhance critical operations, including medical inventory management (e.g., medications, surgical

tools, and protective equipment). These capabilities help avert supply shortages and improve healthcare delivery. This shared foundation enables large-scale, low-power healthcare deployments [5]. At the same time, their growing connectivity increases exposure to cyber threats, underscoring the necessity of secure edge solutions to protect locally generated data. To address these challenges, edge computing bridges the gap between IoT data generation and real-time decision-making by enabling local analytics [6]. It also reduces latency and enhances data privacy, a critical requirement in healthcare and industrial ecosystems. Despite these advantages, the lack of standardized edge computing frameworks hampers interoperability across diverse IoT systems [7], [8].

Another significant challenge is the limited computational, memory, energy, and bandwidth resources of IoT devices. These constraints, combined with the large-scale connectivity

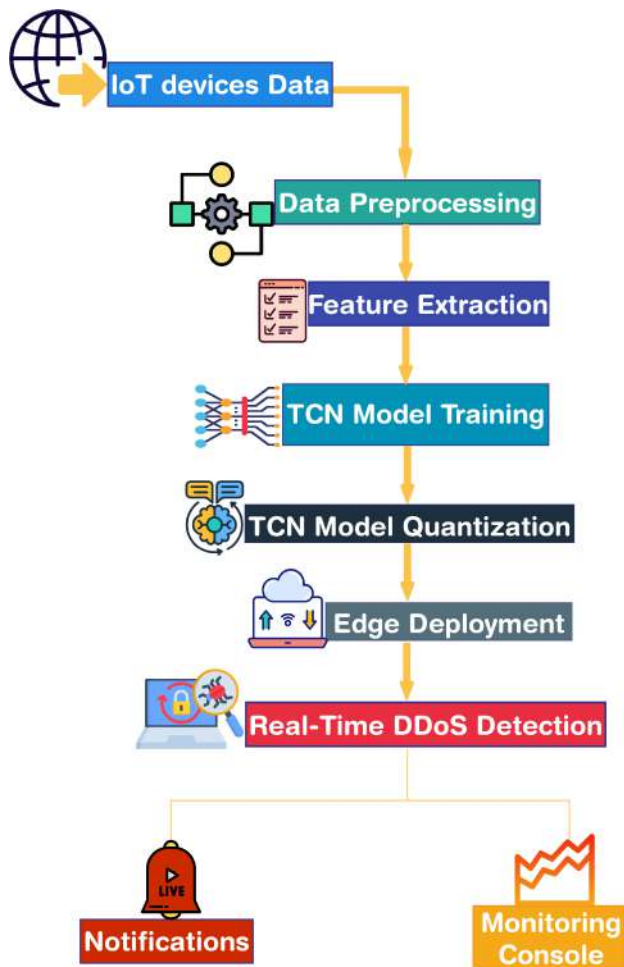


FIGURE 1. Real-time DDoS detection workflow within IoT environments using a TCN model deployment on edge devices.

enabled by 5G networks, intensify the risk of cyber attacks. While 5G extends intelligence closer to the edge and enhances data transmission speed, it also enlarges the attack surface due to the massive number of connected devices [9]. In particular, Distributed Denial of Service (DDoS) attacks pose a severe threat, directly compromising data availability and overwhelming IoT nodes [3], [10]. Securing these systems against DDoS attacks and achieving cyber-resilience remain pressing challenges [2], [11]. Despite significant research efforts, traditional cloud-based security solutions (e.g., Intrusion Detection Systems (IDSs) and centralized authentication) introduce high latency and excessive bandwidth consumption [12]. Consequently, such cloud-based approaches are unsuitable for real-time DDoS detection in critical infrastructures [8], [13].

Given the limitations of cloud-based solutions, edge-based security mechanisms have emerged as an alternative [14]. These approaches enable low-latency processing and local detection of cyber attacks, thereby eliminating reliance on cloud resources. However, deploying deep learning (DL)-based security solutions on edge devices remains challenging

due to limited computational power and constrained memory resources. DL models are resource-intensive, potentially exceeding device capacity, while frequent data transfers between memory and the CPU increase latency and power consumption. Recent studies have explored several DL models, such as Long Short-Term Memory (LSTM) networks [15], Feedforward (FF) models [16], Gated Recurrent Units (GRUs) [17], and Convolutional Neural Networks (CNNs) [18], [19], as well as traditional machine learning (ML) techniques [20], to strengthen IoT security. Despite this progress, most existing approaches incur high computational overhead, making them unsuitable for execution on edge devices. Typically, neural network models operate with 32-bit floating-point (FP32) precision for weights and activations. While this representation ensures high accuracy, it also demands substantial memory and computational resources, making deployment on low-power devices impractical. Thus, reducing model size is not merely an efficiency concern; it is a prerequisite for practical deployment across a wide range of IoT devices.

To address this challenge, this research adopts a DL model, the Temporal Convolutional Network (TCN) [1], which is highly amenable to edge deployment. The reuse and adaptation of previously implemented architectures align with current research trends emphasizing efficient model design for resource-constrained or edge-oriented AI systems [21]. Owing to the TCN model's lightweight nature, quantization reduces precision from FP32 to lower-bit representations (e.g., INT8), thereby minimizing resource consumption while preserving accuracy. INT8 quantization is a widely used standard in deep learning and edge deployment, particularly for TensorFlow Lite (TFLite) and ARM-based platforms [22], [23]. This enables efficient cyberattack classification with high accuracy, low latency, and reduced power usage on edge devices. To evaluate the proposed approach, this research leverages recently published datasets, *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* [24], [25], on resource-constrained edge platforms. The TCN model is converted into TFLite, an optimized framework for efficient deployment of deep learning models on embedded and mobile devices. During conversion, quantization is applied, resulting in a lightweight model that enables practical DDoS detection on constrained IoT devices. Specifically, Raspberry Pi 4 is selected as the edge platform, a low-power ARM-based device widely used in IoT environments [26], [27], [28]. Thus, the primary objective is to develop a practical deployment method for securing IoT systems against DDoS attacks across IoT, H-IoT, and IIoT environments. An overview of the workflow for deploying TCN-based DDoS detection on edge devices is depicted in Figure 1. The performance and feasibility are evaluated on Raspberry Pi 4, including power consumption, inference time, parameter count, and model size. A USB power meter is used to measure power consumption, as software-based profiling is unreliable on

the Raspberry Pi [29], [30]. The key contributions of this research are as follows:

- To the best of our knowledge, this is the first lightweight TCN model deployed on Raspberry Pi 4 for real-time DDoS detection, supporting secure operation in diverse IoT domains, including healthcare and industrial systems. The deployment achieves a reduced parameter count and a smaller model size, making it feasible for IoT devices that have limited RAM while maintaining consistent performance, high accuracy, and low latency on the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets.
- The deployment is practically validated through USB power meter measurements on a Raspberry Pi 4, demonstrating average power consumption across five runs of 4.22 W and 4.64 W for the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets, respectively.

This paper is organized as follows. Section II reviews related work on binary classification and edge deployment in IoT, H-IoT, and IIoT environments. Section III describes the deployment of a lightweight TCN model on Raspberry Pi 4 for efficient DDoS attack classification across diverse IoT settings. Section IV presents a comparative analysis with prior studies, examines the power consumption of the edge system, and reports results on inference time, parameter count, model size, and the confusion matrix. Finally, Section V summarizes the contributions and discusses future research directions.

II. RELATED WORK

This section reviews prior approaches, highlighting key contributions and the limitations that affect the feasibility of edge deployment. In this context, anomaly detection has been widely applied for DDoS detection and mitigation across IoT, H-IoT, and IIoT environments. As highlighted in Section I, several DL models are used for securing IoT environments. Beyond these, hybrid architectures such as CNN-BiLSTM and CNN-LSTM have also been employed in prior studies [19], [31]. These hybrid models introduce a large number of trainable parameters, which diminishes their suitability for deployment scenarios [16]. Moreover, many studies rely on legacy datasets (e.g., CIC-DOS2017, CIC-IDS2017, CIC-IDS2018, NSL-KDD, KDD99, UNSW-NB15) that fail to capture realistic IoT traffic and evolving threat patterns, leading to limited generalization [32], [33], [34]. A summary of recent studies and their main contributions is provided in Table 1.

Akar et al. [15] introduce an LSTM-based intrusion detection model tailored for IoMT environments. Using the CICIoMT2024 dataset, the model achieves a classification accuracy of 98%, demonstrating the effectiveness of deep learning in strengthening IoMT security. Despite this impressive accuracy, the approach suffers from low computational efficiency, which constrains its scalability in practical IoMT deployments. The authors acknowledge that the LSTM

model incurs significant computational overhead, which raises concerns about its practicality and energy efficiency in resource-constrained IoMT environments. Similarly, another study indicates that LSTM-based intrusion detection techniques are constrained by computational complexity and substantial resource requirements [37].

Akhi et al. [1] present a monitoring frequency-based detection and dynamic threshold mitigation method using TCNs for securing 5G H-IoT environments. The approach analyzes each H-IoT node's message count as a percentage of total traffic and dynamically adjusts the DDoS detection and mitigation threshold. The study introduces MQTT and UDP-based datasets using the Cooja and ns-3 simulators, achieving 99.99% accuracy with the TCN model. These datasets are now publicly accessible [24], [25]. Although the method achieves high accuracy for DDoS detection and mitigation, its lack of edge deployment analysis leaves a crucial practical gap that this research aims to address.

In [16], a hybrid LSTM-FF neural network is proposed for detecting low-rate DoS (LR-DoS) attacks. Random Forest is used for feature selection, and the model is evaluated on CIC-DOS2017, CSE-CIC-IDS2018, and LR-HR-DDOS2024 datasets. The approach achieves high accuracy in both binary and multi-class classifications. However, the authors explicitly report that the model has limitations in terms of computational complexity and memory requirements [16]. This limitation is consistent with prior findings that LSTM-based architectures inherently require more computational resources than feedforward neural networks [38].

In [20], the NSL-KDD dataset is used to evaluate three traditional ML methods, namely Random Forest (RF), Logistic Regression (LR), and Naïve Bayes (NB), achieving high accuracy. The study also emphasizes that these models are scalable and efficient for DDoS detection in IoT networks. However, despite achieving high accuracy, such approaches often incur significant inference latency, making them unsuitable for resource-constrained IoT devices. The lack of deployment or validation on actual IoT devices (e.g., ARM-based hardware) further raises uncertainty about their effectiveness in real-world settings. This represents a significant gap, as deployment and latency-aware evaluation are essential to ensure suitability in IoT environments.

In [18], CNNs are applied to anomaly detection in time-series data for H-IoT environments. The study simulates DDoS attacks in the Cooja simulator, which emulates environmental sensors, including temperature and humidity. The CNN-based approach achieves 92% accuracy, indicating its effectiveness in detecting anomalies within H-IoT systems. To further support DDoS detection research, the authors also construct the WSN-DDoS-Attack-H-IoT2023 dataset [36]. However, the study does not examine the model's robustness under diverse DDoS scenarios, including MQTT-based attacks.

In [19], a deep learning-based DDoS detection model for IoT networks is presented. The study employs the CICIDS2017 dataset, which provides realistic traffic with

TABLE 1. Survey of existing DDoS detection methods, attack types, benefits, limitations, and edge deployment in IoT, H-IoT, and IIoT.

Ref.	Technique Used	Targeted Attacks	Key Benefits	Limitations	Edge Deployment
[15] (2025)	LSTM	DoS, DDoS, and Spoofing in IoMT	- 98% accuracy in intrusion detection. - Enhances IoMT security with DL.	- Lacks edge deployment feasibility. - High computational cost for real-time detection.	✗
[1] (2025)	TCN	DDoS in H-IoT	- 99.99% accuracy in DDoS detection and mitigation. - Real-time attack identification by monitoring frequency. - Two H-IoT datasets are generated with Cooja and ns-3 simulator [24], [25].	- Focuses only on DDoS attack detection. - Edge deployment is not investigated.	✗
[16] (2025)	Hybrid LSTM-FF	LR-DoS	- Evaluates performance on multiple datasets; binary and multi-class classification.	- High computational complexity and memory requirements.	✗
[20] (2025)	RF, LR, and NB	DDoS in IoT	- Achieves high accuracy using traditional ML models. - Feature selection using Extra Trees Classifier (ETC).	- Uses older NSL-KDD dataset [35]; no deployment on IoT devices; - no latency analysis; traditional ML models add resource overhead.	✗
[18] (2024)	CNN	DDoS in H-IoT	- Accurate anomaly detection with high accuracy. - Authors create a new dataset [36].	- Specific to environmental IoT sensor data.	✗
[19] (2023)	CNN-BiLSTM	DDoS in IoT	- Demonstrates high accuracy and precision; CNN-BiLSTM is evaluated on multiple datasets (e.g., CICIDS2017, CIDDDoS2019).	- Large model size; high latency; reliance on older datasets; no deployment or discussion on resource-constrained devices (e.g., Raspberry Pi).	✗
[31] (2023)	CNN, LSTM, CNN+LSTM	DoS, Worms, Backdoor, Fuzzers, etc.	- Hybrid CNN-LSTM for IIoT intrusion detection; binary and multi-class classification with up to 99.80% accuracy.	- Datasets used (UNSW-NB15, X-IIoTID) do not explicitly cover DDoS attacks; no latency or efficiency analysis; high model complexity; not evaluated on resource-constrained IIoT/edge devices.	✗

both benign flows and contemporary attack patterns. A CNN with Bidirectional Long Short-Term Memory (CNN-BiLSTM) model is introduced, achieving high performance with an accuracy of 99.76% and a precision of 98.90%. While the results demonstrate effective detection, the centralized evaluation limits insight into the model's scalability and feasibility in distributed IoT settings. Moreover, the study [19] overlooks latency and computational efficiency,

which become significant constraints when the same CNN-BiLSTM architecture is implemented on the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset. This results in higher latency and a larger parameter count than the adopted lightweight TCN model.

In [31], the authors present a deep learning-based intrusion detection approach in IIoT environments. It utilizes the UNSW-NB15 and X-IIoTID datasets, covering both binary

and multi-class classifications. Three models are implemented, including CNN, LSTM, and a hybrid CNN+LSTM, with the hybrid architecture achieving the best performance. Specifically, the CNN-LSTM model attains 93.21% and 92.9% accuracy for binary and multi-class classification on the UNSW-NB15 dataset, while it achieves 99.84% and 99.80% accuracy on the X-IIoTID dataset. The lack of latency and computational efficiency analysis raises concerns for IIoT and large-scale manufacturing environments, where high computational overhead or delayed detection may limit the reliability of industrial processes [39]. In comparison, the adopted lightweight TCN model with quantization addresses these limitations by maintaining near-original accuracy (99.98% $\rightarrow$ 99.95%) while reducing latency, model size, and power usage on Raspberry Pi. This optimization makes it suitable for deployment in diverse IoT environments, including healthcare, industrial, and other resource-constrained settings.

III. METHODOLOGY

This section presents the methodology for adapting the lightweight TCN model architecture introduced in our earlier study [1] into TensorFlow Lite to minimize model size and computational overhead. This adaptation enables efficient deployment on edge devices such as the Raspberry Pi 4 while maintaining high accuracy and low inference latency. The methodology consists of three key phases: data preprocessing, TCN model training, and conversion of the trained TCN model to the TFLite format. After conversion, the model is deployed on the edge device for inference evaluation. The training and conversion experiments are conducted in *Google Colab*, a cloud-based environment for developing and testing deep learning models.

A. DATA PREPROCESSING

This research uses the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets [24], [25] for model training and evaluation. These datasets are derived from simulated H-IoT networks (e.g., body temperature, oxygen saturation, and heart rate) using the Cooja and ns-3 simulators and include both normal traffic and DDoS attack scenarios. Related studies often reuse datasets (e.g., CIC-IDS2017) in the H-IoT domain, even though those datasets are not generated from H-IoT-specific devices [40]. In this context, these previously developed datasets serve as representative benchmarks for evaluating DDoS detection in IoT, H-IoT, and IIoT environments.

The preprocessing pipeline, executed before model training and subsequent TFLite generation, includes transforming features to numeric values, removing rows containing NaN or $\pm\infty$, label encoding of the outcome, a stratified train-test split, and z-score standardization using a *StandardScaler* fitted on the training set and saved for reuse. To ensure numerical stability, rows with invalid entries are removed. As a result, the *UL-ECE-UDP-DDoS-H-IoT2025* dataset has slightly fewer test samples than reported

TABLE 2. Class distribution of the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset.

Split	Class	#Samples	Total
Train (Original)	Malicious	7782	14056
	Normal	6274	
Test	Malicious	3348	6024
	Normal	2676	
Total Dataset Size: (20080, 4, 1)			

TABLE 3. Class distribution of the *UL-ECE-UDP-DDoS-H-IoT2025* dataset (post-preprocessing).

Split	Class	#Samples	Total
Train (Original)	Malicious	35355	69920
	Normal	34565	
Test	Malicious	14370	29967
	Normal	15597	
Total Dataset Size: (99869, 13, 1)			

in earlier research [1] because some records in the raw test split are discarded during preprocessing, as detailed in Table 3. In contrast, the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset contains no such invalid entries, and its test set remains unchanged, as shown in Table 2. The fitted scaler is saved to ensure consistent preprocessing across model training and Raspberry Pi inference. This approach prevents data leakage between the training and test sets.

The Synthetic Minority Oversampling Technique (SMOTE) is applied to the training set to address class imbalance, consistent with the earlier approach [1]. In this method, SMOTE generates synthetic samples for the minority class (Normal) to balance the training distribution. Finally, the preprocessed samples are reshaped into the three-dimensional format required by Conv1D layers, ensuring compatibility with the model input during training and subsequent TFLite conversion.

The conversion workflow incorporates two key refinements to ensure compatibility with TFLite deployment on Raspberry Pi. First, the feature order, scaler, and label encoder are saved as artifacts to ensure consistent preprocessing during real-time inference. Second, a flattened two-dimensional version of the training data is retained for INT8 quantization calibration, required during conversion but not in the original TCN model.

B. TCN MODEL TRAINING

The lightweight TCN model, adapted from our earlier study, is trained to detect DDoS attacks across IoT, IIoT, and H-IoT systems. The architecture consists of two residual blocks, each containing three Conv1D layers with 64 filters, kernel size 3, dilation rates of 1, 2, and 4, and causal padding. The depth value of 3 represents the three dilation levels (1, 2, and 4) used within each residual block, resulting in a total of six Conv1D layers. The input feature matrix

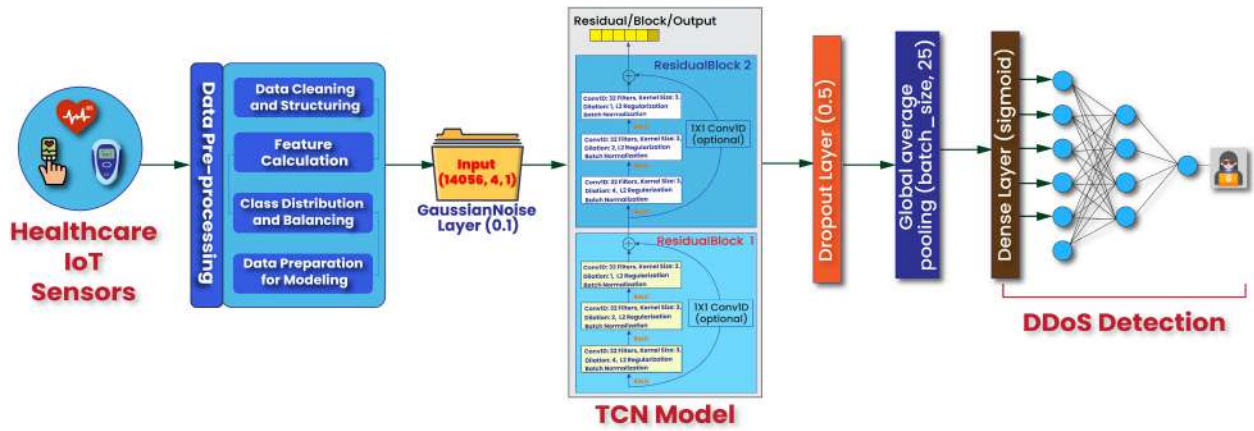


FIGURE 2. Architecture of the lightweight TCN model for DDoS detection within IoT environments, including H-IoT and IIoT. This diagram is adapted from [1] to illustrate the baseline TCN model architecture employed in this paper.

TABLE 4. Model configuration and training parameters of the lightweight TCN model.

Components and Hyperparameters	Value
Number of features (MQTT)	4
Number of features (UDP)	13
Number of classes	2 (Normal and Malicious)
Number of residual blocks	2 blocks, each with 3 convolutional layers
Filter count per TCN layer	64
Convolution kernel size	3
Dilation rates applied	1, 2, 4
Type of padding used	Causal (preserves temporal order)
Batch normalization effect	Improves stability and convergence
Model depth (number of stacked TCN layers)	3
Optimizer used	Adam
Learning rate applied	5e-5
Loss function	Binary cross-entropy
Batch size	64
Epochs	50
Regularization techniques	Dropout (0.2 - ReLU, 0.5 - Sigmoid), L2 regularization (0.01)
Pooling Layer	Global average pooling
Input Shape	$n_{samples} \times n_{features} \times n_{timesteps}$
Output Layer	Dense layer with Sigmoid activation
Noise injection method	GaussianNoise (0.1)
Validation set ratio	20%

Note: This table presents the model configuration, activation functions, learning strategy, and other hyperparameters used in the lightweight TCN model for DDoS detection on the MQTT and UDP-based datasets. In this implementation, each feature vector corresponds to a single time step, with $n_{timesteps} = 1$, which reduces the input shape to $(n_{samples}, n_{features}, 1)$.

X is reshaped to $(N, F, 1)$ so it can be processed by 1D convolutional layers. Since each traffic record is treated as an independent feature vector, the input format consists of one feature dimension and one channel. The identical reshaping is applied consistently during training and before TFLite conversion for deployment on the Raspberry Pi, ensuring that the Conv1D layers process the input correctly. This design expands the receptive field across multiple time scales while maintaining computational efficiency. Causal padding ensures that predictions at time t are conditioned

only on inputs up to t , thereby preserving temporal causality. Batch Normalization is applied after each convolutional layer to stabilize training and improve convergence. To further reduce overfitting, ℓ_2 regularization with a factor of 0.01 is applied to all convolutional layers. A Gaussian noise layer ($\sigma = 0.1$) is added at the input, and a Dropout layer (rate 0.5) is placed after the second residual block. As in [1], the TCN architecture uses global average pooling for temporal aggregation, followed by a sigmoid-activated dense layer for binary classification. In this paper, the

architecture is unchanged; the focus is on its quantized conversion and deployment on low-power edge devices. For deployment, this pooling operation is implemented using TensorFlow's built-in GlobalAveragePooling1D layer, which is supported by the model conversion pipeline and enables stable INT8 export. Model training uses the Adam optimizer (learning rate 5×10^{-5}), a batch size of 64, and up to 50 epochs, with early stopping and learning rate reduction on plateau. This architecture captures temporal dependencies across IoT, H-IoT, and IIoT traffic, enabling effective DDoS classification, as detailed in Table 4 and illustrated in Figure 2.

C. TCN MODEL CONVERSION INTO TFLITE

The TCN model, trained on the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets, is converted into TFLite format to enable efficient deployment on resource-constrained edge devices such as the Raspberry Pi 4.

For deployment, the trained TCN is converted using the `TFLiteConverter` with TensorFlow Lite's default optimization strategy (DEFAULT), which applies post-training optimizations such as weight quantization and computational graph simplifications. Full-integer (INT8) quantization is enforced by restricting the converter to integer kernels and setting both input and output types to INT8. In this configuration, all convolutional, batch normalization, pooling, and dense layers are fully quantized with INT8 weights and activations. A representative subset of up to 300 samples from each dataset (drawn from the scaled data splits) calibrates activation ranges; a few hundred samples are sufficient to capture activation distributions while maintaining efficient conversion. For each dataset, the complete held-out test split is exported as `test.csv` to enable reproducible inference directly on the Raspberry Pi during evaluation.

This quantization reduces model size by approximately $4\times$, since each parameter requires 1 byte in INT8 instead of 4 bytes in FP32. The converted INT8 models are saved, and their real-world file sizes are reported alongside the theoretical parameter-size reduction. In addition, the exported TFLite model uses INT8 for both input and output tensors, ensuring that runtime execution on the Raspberry Pi 4 relies on integer arithmetic rather than FP32 operations. During conversion, TensorFlow Lite serializes the model into its FlatBuffer format, a compact binary structure for storing network graphs and parameters [41]. Any remaining non-INT8 buffers correspond to constants folded through graph optimization (e.g., bias terms and fused batch-normalization parameters) and therefore do not trigger floating-point computation during runtime.

Although the scale and zero-point are estimated internally by TensorFlow Lite during post-training quantization, the process follows standard affine quantization. For signed 8-bit

quantization,

$$q = \text{clip}(\text{round}(f/s) + z, q_{\min}, q_{\max}), \quad f \approx s(q - z),$$

with $q_{\min} = -128$ and $q_{\max} = 127$. The scale and zero-point are computed as

$$s = \frac{\max(F) - \min(F)}{q_{\max} - q_{\min}}, \quad z = \text{round}\left(q_{\min} - \frac{\min(F)}{s}\right),$$

where F denotes the range of floating-point values.

In practice, the zero-point is clamped to the valid range $[q_{\min}, q_{\max}]$, and thus remains within $[-128, 127]$ [42], [43].

During evaluation, full INT8 quantization of the TCN model results in a minimal change in DDoS detection accuracy compared to the FP32 model. All evaluation metrics remain within $\pm 0.05\%$ of the original FP32 values on both the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets. This indicates that quantization preserves detection performance while improving deployment efficiency on the Raspberry Pi 4.

D. REAL-TIME DEPLOYMENT OF TCN ON RASPBERRY PI 4

The TCN TFLite model is deployed on a Raspberry Pi 4 to detect DDoS traffic in IoT settings, using the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets. The deployment consists of three steps: loading and preprocessing the dataset, loading the TFLite model, and performing inference on the edge device. The end-to-end workflow is shown in Figure 3.

1) DATASET LOADING AND PREPROCESSING

During deployment, both *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* test datasets are loaded on the Raspberry Pi and preprocessed following the same procedure used during the original TCN model training to ensure consistency. The feature schema, extracted from the training data and saved before TFLite conversion, is reapplied to preserve the expected feature set and order for input alignment. All feature values are coerced to numeric, $\pm\infty$ entries are converted to NaN, and rows containing NaN values are discarded. The saved `StandardScaler` and feature schema from training are applied on the Raspberry Pi before inference. SMOTE is not used during inference. If the outcome column is present, it is excluded from scaling and retained separately for evaluation. The processed data are reshaped to represent each sample as a sequence of input features with a single channel, as required by the Conv1D layers of the TCN model. In this configuration, the number of features is 4 for *UL-ECE-MQTT-DDoS-H-IoT2025* and 13 for *UL-ECE-UDP-DDoS-H-IoT2025*.

2) LOADING THE TFLITE MODEL

The TFLite runtime is a lightweight interpreter that executes the converted model on-device. It is configured with full INT8 input/output quantization to reduce computational cost and enable efficient execution on edge hardware

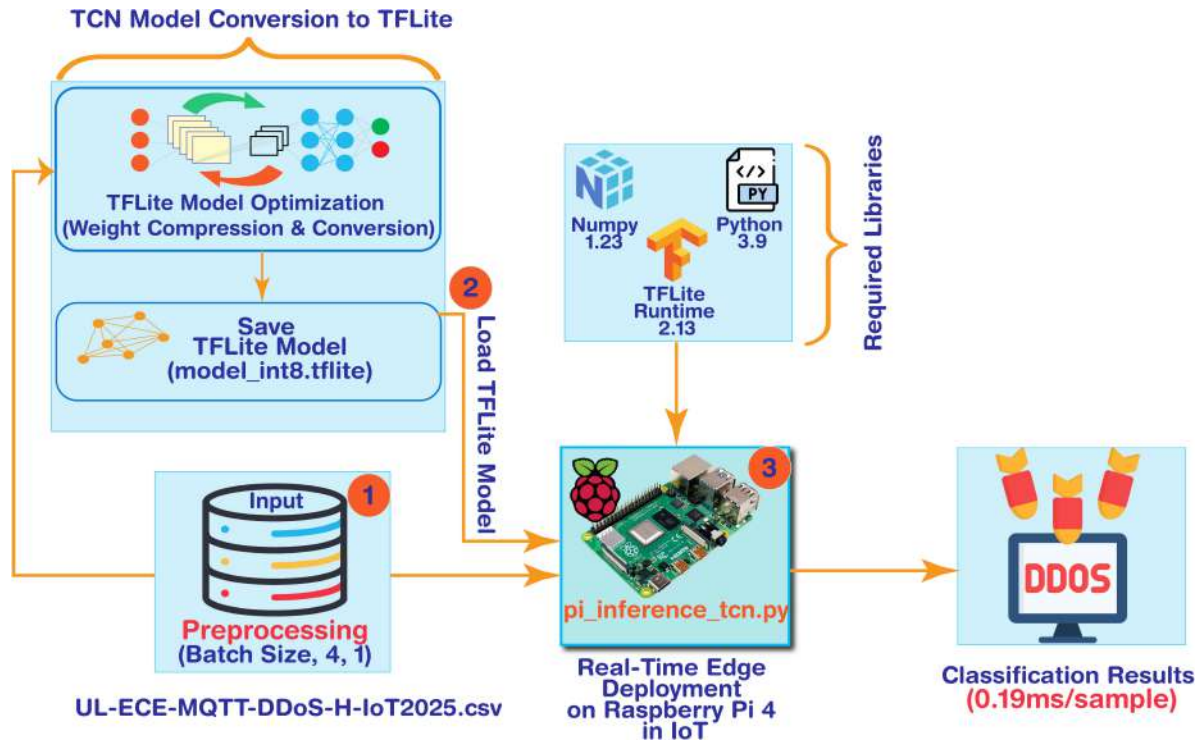


FIGURE 3. Real-time deployment of the lightweight TCN model on Raspberry Pi 4. The process involves converting the model to TFLite, performing data preprocessing, and enabling real-time inference and classification.

such as the Raspberry Pi 4. XNNPACK delegation is enabled when supported to reduce inference latency, and the number of inference threads is configured according to the available CPU cores (e.g., 1–4 on a Raspberry Pi 4). To prepare the model for inference, the input tensor is explicitly resized to match the expected shape $[1, F, 1]$ before `allocate_tensors()` because the interpreter requires tensor dimensions to be defined in advance to allocate memory correctly. After allocation, the interpreter metadata is used to verify that the input and output tensors are in INT8 format for each dataset. To ensure reproducibility, random seeds are fixed, and the model's declared feature count is validated against the preprocessed dataset.

3) INFERENCE AND CLASSIFICATION

Inference is performed sample by sample (batch = 1). Each normalized feature vector is quantized to INT8 using the interpreter's input scale and zero-point parameters and then provided to the TFLite interpreter for inference. The output is subsequently dequantized to floating point using the corresponding output parameters. A threshold of 0.5 is applied to classify each sample as {Normal, DDoS}. When true class labels (ground truth) are available, the saved label encoder is used to map them, and evaluation metrics (accuracy, precision, recall, F1-score) are reported. To avoid inflated latency from one-time initialization effects, warm-up inferences are executed and discarded before measurement. In practice, 5–10 warm-up runs are typically sufficient on TFLite; hence, ten are used for the

UL-ECE-MQTT-DDoS-H-IoT2025 dataset and five for the *UL-ECE-UDP-DDoS-H-IoT2025* dataset.

IV. PERFORMANCE EVALUATION AND COMPARISON

This section presents a detailed evaluation of the adopted lightweight TCN model, comparing it with previously published ML and DL methods across multiple performance metrics. It further reports the layer-wise parameter counts and per-sample inference time on Google Colab. The evaluation of the TCN TFLite model focuses on its inference latency on the Raspberry Pi 4. In addition, power consumption is measured across the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets, capturing temporal variations in power usage. A confusion matrix is presented to illustrate the model's classification performance for DDoS detection on Raspberry Pi 4. It also includes cross-dataset deployment evaluation, edge security considerations, five real-world case studies, security analysis, and attack coverage across diverse IoT environments, and a discussion of TCN model selection.

A. COMPARISON WITH PREVIOUS RESEARCH

The lightweight TCN model is compared with existing ML and DL methods, including LSTM [15], Hybrid LSTM-FF [16], LR [20], CNN-BiLSTM [19], and CNN-LSTM [31], all implemented on the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset, as outlined in Table 5. The results demonstrate that the TCN model achieves higher accuracy, precision, recall, and F1-score than previous approaches. The 95%

TABLE 5. Comparison of the lightweight TCN (TFLite on Pi 4) model with existing ML and DL methods on the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset.

Reference	Method	Protocol	Acc. (%)	Prec. (%)	Rec. (%)	F1 (%)	Params	Inf. time	95% CI
[15] (2025)	LSTM	–	81.29	87.71	77.15	82.09	167,377	0.11	–
[16] (2025)	Hybrid LSTM-FF	–	98.49	99.72	97.54	98.62	182,865	0.24	–
[20] (2025)	LR	–	99.97	100	99.94	99.97	5	0.25	–
[19] (2023)	CNN-BiLSTM	–	99.96	99.95	99.98	99.97	145,121	0.15	–
[31] (2023)	CNN+LSTM	–	99.97	99.97	99.98	99.97	589,953	0.11	–
[1] (2025)	TCN (Original)	MQTT	99.98	99.97	100	99.99	63,617	0.06	–
[1] (2025)	TCN (Original)	UDP	99.99	99.99	99.99	99.99	63,617	0.06	–
Deployed	Lightweight TCN (TFLite on Pi 4)	MQTT	99.95	99.97	99.94	99.96	63,136	0.19	[99.85, 99.98]
Deployed	Lightweight TCN (TFLite on Pi 4)	UDP	99.94	99.88	100	99.94	63,138	0.27	[99.91, 99.96]

Note: Results of prior methods and the Original TCN are obtained on Google Colab. The Lightweight TCN (TFLite on Pi 4) refers to the deployment of the Original TCN model on the Raspberry Pi 4, with separate results reported for the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets. For brevity, the datasets are denoted by their corresponding protocols (i.e., MQTT and UDP). Inference time values are reported in milliseconds per sample (ms/sample).

TABLE 6. Model size and parameter count before and after TFLite conversion of the TCN model for both datasets.

TCN Model	Parameters	File size (KB)
Original FP32 (Keras)	63617	248.5
INT8 TFLite (MQTT)	63136	89.1
INT8 TFLite (UDP)	63138	89.1

Note: The parameter count differs by ± 2 between the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets due to variations in input features. This does not affect model size.

confidence intervals (CIs) for accuracy across both datasets indicate low variability around the mean, with changes of less than 0.1%, as reported in Table 5. These narrow bounds demonstrate that the lightweight quantized TCN model maintains stable accuracy on the Raspberry Pi 4. It also exhibits lower inference time per sample and a reduced parameter count, obtained through layer-wise analysis, thereby enhancing its efficiency for real-time applications across IoT, IIoT, and H-IoT. The average per-sample inference time on the Raspberry Pi 4 is 0.19 ms for the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset and 0.27 ms for the *UL-ECE-UDP-DDoS-H-IoT2025* dataset, as shown in Table 5. As the mean inference time remains identical across the five runs for each dataset, the standard deviation becomes zero, and the confidence interval collapses to identical lower and upper bounds, offering no additional insight. The calculation of the mean inference time is formalized as:

$$\text{Mean Inference Time} = \frac{\sum \text{Inference Times}}{\text{Number of Runs}} \quad (1)$$

where:

- $\sum \text{Inference Times}$ represents the total cumulative inference time for all processed samples.
- Number of Runs corresponds to the total number of inference executions.

Compared to the original TCN, the lightweight TCN TFLite version exhibits slightly lower accuracy after quantization. Nevertheless, the parameter count decreases after converting the original TCN to its TFLite version. This reduction is beneficial for edge deployment, as it lowers memory requirements and reduces power-consumption overhead on resource-constrained devices. Notably, the parameter count varies slightly between the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets, as shown in Table 5. This arises from differences in the dimensions of input features after preprocessing, which affect the size of the first convolutional layer. Since the *UL-ECE-UDP-DDoS-H-IoT2025* dataset includes more features in the input layer, it results in slightly different weight shapes and a marginally higher parameter count (two additional parameters) compared to the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset, as shown in Table 6. This negligible increase does not affect the overall TFLite model size. Such accuracy degradation is common in quantized models, particularly when deployed on resource-constrained hardware [44]. On the other hand, inference time can vary significantly across different platforms (e.g., Google Colab vs. Raspberry Pi 4) due to differences in system architecture, operating systems, and resource availability. Specifying the computing environment is a standard practice in edge AI benchmarking to ensure consistent results and fair comparison [45]. Therefore, the TCN TFLite model is well-suited for low-power edge devices, balancing accuracy, efficiency, and resource consumption.

B. POWER CONSUMPTION ANALYSIS

The power consumption over time is measured by video recording the USB power meter connected to the Raspberry Pi 4 during execution of the TCN TFLite model on both datasets, namely *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025*, as shown in Figure 4. Before the inference scripts are executed, the Raspberry Pi 4 remains in an idle state, drawing a stable baseline

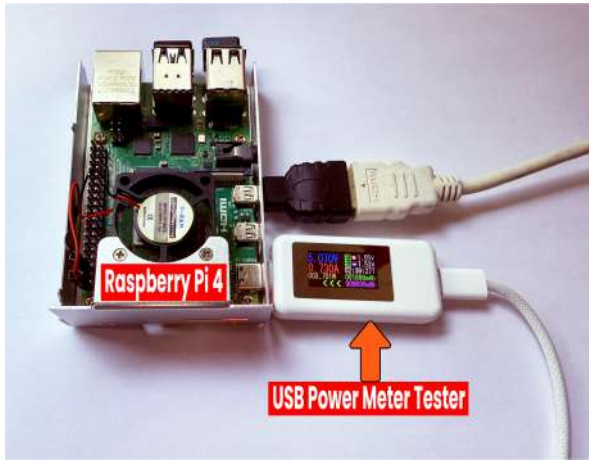


FIGURE 4. Raspberry Pi 4 with USB Power Meter Tester for real-time power measurement during model inference.

power (≈ 3.7 W). This baseline reflects the power required to keep the board powered, maintain Raspberry Pi OS (64-bit) based on Debian 11 (Bullseye), and support background services, even when no inference model is executing. For both datasets, recording begins with the Raspberry Pi 4 in an idle state, and the inference script is executed at the start of the video. Recording stops when the inference completion message appears in the terminal. Consequently, the total recorded duration varies slightly between runs (typically within ± 1 s) due to manual start/stop timing and minor system delays. The same inference procedure is followed across all five runs. The subsequent analysis focuses on the inference execution time for each dataset.

1) UL-ECE-MQTT-DDoS-H-IoT2025 DATASET

For execution on the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset, power readings start at 0.06 s in all runs, while the ending time differs across the five runs and reaches a maximum of 11.41 s. Run 1 contains 11 time–power samples, Runs 2 and 3 report 12 samples, and Run 4 consists of 13 samples, even though its ending time (11.06 s) is shorter than that of Run 2. This difference occurs because power values are recorded whenever the USB power meter display changes, whereas additional stable values are manually recorded rather than sampled at fixed time intervals. For example, in Run 4, the displayed power remains at 5.186 W from 8.50 s onwards, and these unchanged readings are included to capture the constant-power phase before termination. As a result, runs with more power fluctuations generate more recorded samples, even if the total duration is similar. This variation in sampling does not affect the energy calculation, as trapezoidal numerical integration accounts for non-uniform time spacing [46]. All recorded time–power data points for each run are listed in Table 7.

The power profile of the Raspberry Pi 4 during execution of the TFLite model on the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset is illustrated in Figure 5, where each curve corresponds to one of five independent runs. Power remains

close to the idle level (approximately 3.66 W) during the initialization phase, then rises rapidly starting at approximately 6 s and reaching its peak around 7 s as inference begins. After this transition, power stays in a high-power plateau of roughly 4.9–5.2 W for the remainder of each run, corresponding to the active inference phase. Peak power and minimum power values are calculated individually for each run and then averaged across the five runs, as summarized in Table 8. The dashed horizontal line denotes the mean power consumption of 4.225 W, while the shaded region represents the 95% confidence interval [4.152, 4.299]. The coefficient of variation (CV), defined as the standard deviation divided by the mean, is 1.40%, indicating low relative variability across the five runs.

Moreover, a statistical summary of the Raspberry Pi 4 power measurements on the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset across five experimental runs is provided, including average execution duration, average total energy, average power, peak power, and minimum power, as detailed in Table 8. The average execution duration across the five runs is 10.738 s with a standard deviation of 0.515 s. The total energy consumption for each run is computed from the measured power–time samples. The mean energy across the five runs is 45.379 J, representing the time-integrated electrical demand of a single inference execution, expressed in joules (J). For each run, the average power over its execution duration is computed from the corresponding power–time samples. These five run-wise average power values are then used to calculate the aggregate statistics, including the overall mean, standard deviation, CV, and 95% confidence interval. The standard deviation of the average power across the five runs is 0.059 W, and the CV is 1.40%. The column $n = 5$ indicates that all reported statistics are computed using five separate experimental runs. The reported 95% confidence intervals quantify the uncertainty in the estimated means across these runs. While Figure 5 presents instantaneous power trajectories over time, Table 8 summarizes run-level aggregate metrics. The analysis of the video-derived time data, together with the measured power values, outlines the system’s power trajectory during execution and transition toward idle. Based on this trajectory, energy consumption is calculated using the trapezoidal rule, which integrates the discrete time–power measurements [46], [47].

2) UL-ECE-UDP-DDoS-H-IoT2025 DATASET

The power measurements for the *UL-ECE-UDP-DDoS-H-IoT2025* dataset follow the same procedure as the MQTT experiments. The first time-stamped sample across the five runs lies between 0.05 s and 0.08 s, as shown in Table 9. This offset arises because the power meter and camera operate on independent refresh cycles, so the first recorded frame may not align with a meter update. All runs except Run 5 start near 3.7 W (3.70–3.72 W), while Run 5 starts slightly higher at 3.91 W before it converges to the same range; this phase corresponds to the script initialisation

TABLE 7. Power consumption measurement data points across five runs for the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset.

Run 1		Run 2		Run 3		Run 4		Run 5	
Time (s)	Power (W)	Time (s)	Power (W)	Time (s)	Power (W)	Time (s)	Power (W)	Time (s)	Power (W)
0.06	3.676	0.06	3.671	0.06	3.666	0.06	3.671	0.06	3.677
1.02	3.676	1.00	3.671	0.17	3.650	1.00	3.671	1.00	3.677
2.34	3.677	2.00	3.671	1.20	3.650	2.00	3.671	2.00	3.677
3.41	3.661	2.87	3.645	2.30	3.650	3.11	3.645	2.80	3.650
4.50	3.661	3.90	3.645	3.40	3.651	4.20	3.645	3.90	3.650
5.50	3.661	5.00	3.645	4.48	3.661	5.30	3.645	5.00	3.650
6.64	4.674	6.00	3.645	5.30	3.661	6.36	4.605	6.01	4.264
7.71	4.935	7.04	5.032	6.62	4.451	7.37	5.185	7.07	5.196
8.80	4.935	8.20	4.892	7.67	5.111	7.41	5.187	8.14	4.899
9.90	4.935	9.20	4.892	8.70	5.111	8.50	5.186	9.10	4.899
10.91	4.901	10.30	4.892	9.60	5.111	9.50	5.186	10.06	4.899
–	–	11.41	4.564	10.55	5.111	10.61	5.186	–	–
–	–	–	–	–	–	11.06	5.186	–	–

Note: Recording start and stop occur manually based on the terminal completion message. Therefore, the total measurement duration and the number of recorded samples vary slightly across runs (typically within ± 1 s). Cells marked “–” indicate that no further power sample is available for that run because the corresponding trace finishes earlier than the others.

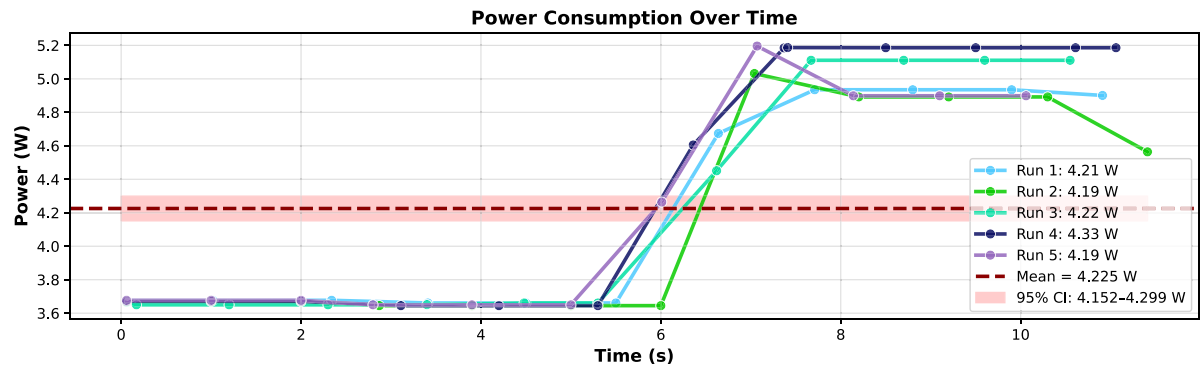


FIGURE 5. Instantaneous power consumption of the Raspberry Pi 4 during execution of the TCN TFLite model on the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset. The five run-level traces are shown alongside the mean power value and its 95% confidence interval.

TABLE 8. Statistical summary of Raspberry Pi 4 power measurements over five separate runs on the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset.

Metric	Mean	Std Dev	CV (%)	95% CI	n
Average Duration (s)	10.738	0.515	4.80	[10.098, 11.378]	5
Average Total Energy (J)	45.379	2.393	5.27	[42.407, 48.351]	5
Average Power (W)	4.225	0.059	1.40	[4.152, 4.299]	5
Peak Power (W)	5.092	0.110	2.16	[4.956, 5.229]	5
Minimum Power (W)	3.650	0.007	0.18	[3.642, 3.658]	5

Note: All statistics are computed from five experimental runs ($n = 5$). The average execution duration and average total energy are first computed for each run and then aggregated across the five runs. Peak and minimum power correspond to the maximum and minimum instantaneous power values observed in each run, averaged across runs. The reported standard deviation and CV describe the dispersion of each metric, while the 95% confidence interval reflects the uncertainty of the estimated mean values.

period. The TCN TFLite inference script enters its active processing phase approximately between 6 s and 9 s across the five runs, marked by a distinct rise in power from the idle level to the 5.0–5.3 W range. After this transition, the traces stabilise into a steady-power plateau ranging approximately from 4.82 W to 5.06 W for the remainder of the execution window. Toward the end of each run, power decreases as the

inference script completes and the Raspberry Pi 4 returns to its idle level.

Moreover, the run-specific fluctuations in the idle region, transient spikes at the start of computation, and variations around the transition step are illustrated in Figure 6. It also includes the per-run average, the mean power across five runs of 4.640 W, and the 95% confidence interval [4.56,

TABLE 9. Power consumption measurement data points across five runs for the *UL-ECE-UDP-DDoS-H-IoT2025* dataset.

Run 1		Run 2		Run 3		Run 4		Run 5	
Time (s)	Power (W)	Time (s)	Power (W)	Time (s)	Power (W)	Time (s)	Power (W)	Time (s)	Power (W)
0.05	3.707	0.07	3.701	0.08	3.717	0.06	3.723	0.05	3.910
1.00	3.707	1.11	3.696	1.10	3.717	1.10	3.723	1.10	3.739
2.00	3.707	2.20	3.696	2.20	3.701	2.10	3.670	2.20	3.739
3.14	3.697	3.30	3.696	3.27	3.702	3.20	3.670	3.30	3.739
4.20	3.697	4.40	3.696	4.30	3.702	4.30	3.670	4.30	3.727
5.30	3.697	5.40	3.728	5.40	3.702	5.40	3.670	5.40	3.727
6.30	4.697	6.50	3.728	6.47	3.686	6.37	3.766	6.44	4.927
6.34	4.289	7.54	3.729	7.53	3.685	7.43	3.763	7.50	5.166
7.40	5.286	8.61	5.217	8.60	3.685	8.50	5.389	8.60	5.166
8.47	4.823	9.68	5.167	9.67	5.019	9.60	4.979	9.70	5.166
9.50	4.823	10.70	5.167	10.76	4.971	10.70	4.979	10.70	5.066
10.60	4.823	11.80	5.167	11.84	4.936	11.74	4.945	10.74	5.024
11.70	4.823	12.90	4.977	12.90	4.936	12.80	4.945	11.80	5.035
12.73	4.870	13.96	4.989	14.00	4.936	13.87	5.019	12.90	5.035
13.80	4.870	15.00	4.989	15.03	5.024	15.00	5.019	14.00	5.035
14.90	4.870	16.10	4.989	16.10	5.024	16.10	5.019	15.10	5.035
15.94	5.024	17.17	5.030	17.20	5.024	17.20	5.019	16.20	5.035
17.00	5.024	18.20	5.030	18.30	5.024	18.13	5.030	17.30	5.035
18.10	5.024	19.30	5.030	19.40	5.024	19.20	5.030	19.26	5.024
19.20	5.024	20.40	5.030	20.50	5.024	20.30	5.030	20.30	5.024
20.30	5.024	21.50	5.030	21.60	5.024	21.40	5.030	21.40	5.024
21.40	5.024	22.60	5.030	22.70	5.024	22.50	5.030	22.50	5.024
22.50	5.024	23.70	5.030	23.60	5.056	23.60	5.030	23.53	5.024
23.60	5.024	24.80	5.030	24.70	5.056	24.56	4.701	–	–
24.51	5.035	25.19	5.030	25.80	5.056	25.03	4.701	–	–
25.60	5.035	–	–	26.90	5.056	–	–	–	–
26.70	5.035	–	–	27.23	4.463	–	–	–	–
27.72	4.435	–	–	–	–	–	–	–	–

Note: Start and stop of each run are controlled manually based on the terminal completion message. Consequently, the total measurement duration and the number of recorded samples differ slightly across runs. Cells marked “–” indicate that no further power sample is available for that run in that row because the corresponding trace finishes earlier than the others.

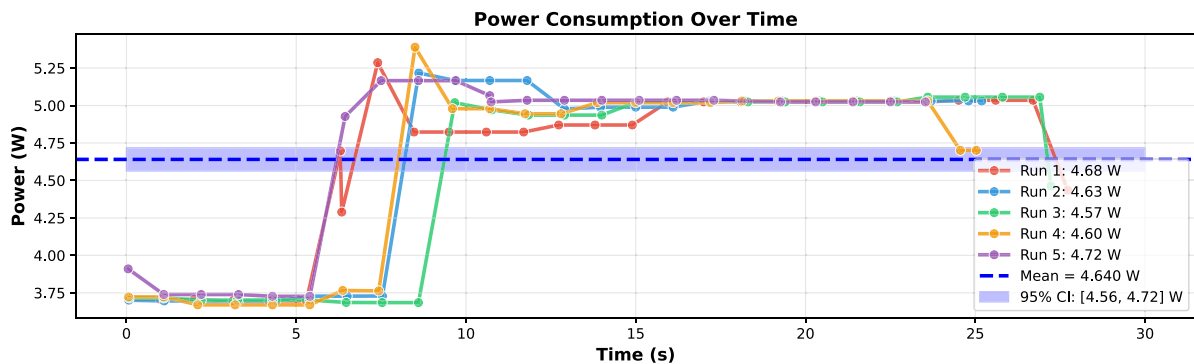


FIGURE 6. Power consumption of the Raspberry Pi 4 while executing the TCN TFLite model on the *UL-ECE-UDP-DDoS-H-IoT2025* dataset, with the average power highlighted.

4.72], which indicates the lower and upper bounds around the mean power during the inference execution phase.

A statistical summary of the Raspberry Pi 4 power consumptions over five runs on the *UL-ECE-UDP-DDoS-H-IoT2025* dataset is listed in Table 10, reporting the average execution duration, total energy, mean power, peak power,

and minimum power. The Raspberry Pi 4 maintains a consistent power profile during active execution, with a low coefficient of variation of 1.34%, indicating that the mean power stays stable across runs. The total energy varies between approximately 109.80 and 128.45 J, primarily due to differences in execution duration. The instantaneous

TABLE 10. Statistical summary of Raspberry Pi 4 power measurements over five runs ($n = 5$) on the *UL-ECE-UDP-DDoS-H-IoT2025* dataset.

Metric	Mean	Std Dev	CV (%)	95% CI	n
Average Duration (s)	25.678	1.716	6.68	[23.547, 27.809]	5
Average Total Energy (J)	119.124	7.511	6.31	[109.798, 128.449]	5
Average Power (W)	4.640	0.062	1.34	[4.563, 4.718]	5
Peak Power (W)	5.223	0.125	2.40	[5.067, 5.378]	5
Minimum Power (W)	3.695	0.021	0.57	[3.669, 3.721]	5

Note: All statistics use five experimental runs ($n = 5$). Duration and total energy values are run-level means. Peak and minimum power correspond to the maximum and minimum instantaneous power observed in each run, averaged across runs. The reported standard deviation and CV describe the dispersion of each metric, while the 95% confidence intervals indicate the uncertainty in the estimated mean values.

power values also remain clustered, with peak levels between 5.067 and 5.378 W and minimum levels between 3.669 and 3.721 W.

A comparison of the two datasets shows a clear difference in sustained power demand during TCN inference. The MQTT-based dataset produces an average active power of 4.225 W, whereas the UDP-based dataset produces 4.640 W, as depicted in Figures 5 and 6. Subtracting the shared idle region near 3.7 W yields an effective computational overhead of approximately 0.53 W for MQTT and 0.94 W for UDP. This higher requirement in the UDP-based dataset aligns with the heavier packet processing observed in its transition and plateau regions. Despite this difference, both workloads maintain a tightly grouped plateau and exhibit stable behaviour across runs, supporting the suitability of the lightweight TCN model for deployment in resource-constrained IoT environments.

To ensure reproducibility, an example USB power meter recording and the corresponding time–power data files for both datasets are publicly available in our GitHub repository [48]. The repository also illustrates the whole measurement procedure to support method transparency, while the paper reports the extracted data from five independent runs.

C. CONFUSION MATRIX ANALYSIS

The confusion matrix is employed to evaluate the binary classification performance of the lightweight TCN model on the Raspberry Pi 4 within IoT environments, including H-IoT and IIoT. It demonstrates the model’s ability to distinguish Normal from DDoS traffic with high accuracy and minimal misclassification.

As shown in (2), the confusion matrix for the *UL-ECE-MQTT-DDoS-H-IoT2025* dataset reports 2675 true negatives (TN), 3346 true positives (TP), 1 false positive (FP), and 2 false negatives (FN). This corresponds to only three misclassifications out of 6024 samples, indicating high classification accuracy.

For the *UL-ECE-UDP-DDoS-H-IoT2025* dataset, the model achieves 14352 TN and 15597 TP, with 18 FP and no FN, as reported in (3). This outcome indicates consistent performance, with few false positives and no false negatives, ensuring that all DDoS-affected nodes are

correctly identified.

$$\text{Confusion Matrix} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix} = \begin{bmatrix} 2675 & 1 \\ 2 & 3346 \end{bmatrix} \quad (2)$$

$$\text{Confusion Matrix} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix} = \begin{bmatrix} 14352 & 18 \\ 0 & 15597 \end{bmatrix} \quad (3)$$

D. DEPLOYMENT FOR CROSS-DATASET EVALUATION

This section presents the deployment of a lightweight TCN on the Raspberry Pi 4 using the public CICIoT2023 and CICIoT2025 datasets, which represent generic IoT and IIoT traffic, respectively. This cross-datasets evaluation assesses model generalization beyond the H-IoT DDoS datasets used in this study.

1) CICIoT2023 DATASET

A binary classification pipeline is designed for cross-dataset evaluation using CICIoT2023 traffic data. The process starts by constructing a DDoS UDP flood traffic dataset. Initially, four UDP flood traffic files are randomly selected from the CICIoT2023 dataset, namely “DDoS-UDP_Flood1.pcap.csv”, “DDoS-UDP_Flood5.pcap.csv”, “DDoS-UDP_Flood10.pcap.csv”, and “DDoS-UDP_Flood15.pcap.csv”. These files are merged to produce a consolidated dataset, “merged_CICIoT2023_DD oSUDP.csv”, which captures varying UDP flood intensities. Two benign traffic samples are subsequently collected from “BenignTraffic.pcap.csv” and “BenignTraffic2.pcap.csv” and concatenated into a single benign dataset. A binary outcome column is introduced to distinguish between normal and attack traffic. Feature column consistency is enforced, and the benign dataset is aligned with the same feature structure as the DDoS UDP dataset. The benign subset is then merged with the consolidated DDoS UDP traffic to construct the final binary dataset named “CICIoT2023_DD oS_vs_Normal_binary.csv”. The combined dataset is randomly shuffled to avoid ordering bias. After merging, duplicate samples are inspected, and all exact duplicate rows are removed to reduce redundancy and prevent sampling bias. The final deduplicated dataset is saved as “CICIoT2023_clean_binary.csv”. A summary of the dataset composition and final feature dimension after preprocessing and leakage removal is presented in Table 11.

TABLE 11. Summary of the CICIoT2023 and CICIoT2025 datasets.

CICIoT2023 Dataset	
Metric	Count
Total samples	1180686
DDoS attack samples	510000
Benign samples	670686
Final feature dimension	38
Training samples (70%)	826480
Testing samples (30%)	354206
CICIoT2025 Dataset	
Total samples	238828
Attack samples	129236
Benign samples	109592
Final feature dimension	71
Training samples (70%)	167179
Testing samples (30%)	71649

During data preprocessing, a leakage analysis is applied to the cleaned dataset. All features are converted to numeric values, and rows containing NaN or infinite values are removed. Label and identifier features such as label, class, attack_type, and is_attack are explicitly checked and removed if present. Timestamp columns are also removed to prevent indirect temporal leakage. Pearson correlation is then computed between each remaining feature and the target label. The “Number” column shows a high correlation with the target (Pearson correlation ≈ 0.999), indicating direct label leakage. This column is removed from both the training and test sets to prevent artificially inflated performance. After these removals, no remaining feature exhibits near-perfect correlation with the target label, confirming the absence of detectable feature-level leakage.

A stratified 70–30 train–test split is applied to preserve class distribution. Feature normalization is performed using a StandardScaler fitted only on the training data. The same scaler and feature order are reused during deployment to maintain preprocessing consistency.

The lightweight TCN model is fully quantized to INT8 using TensorFlow Lite and deployed on the Raspberry Pi 4. The deployment procedure follows the same pipeline used for the MQTT and UDP-based datasets, as described in Section III-D. Evaluation on the CICIoT2023 dataset demonstrates a deployment accuracy of 98.43%, with a precision of 97.13% and a recall of 99.30%. All deployment performance metrics are reported in Table 12.

The confusion matrix shows 1068 false negatives and 151995 true positives, indicating that the model rarely misclassifies attack traffic as normal, which is critical for security-sensitive IoT environments. A total of 4494 false positives is observed, reflecting a bias toward attack detection. This behavior is acceptable in intrusion-detection scenarios, where failing to detect an attack poses a higher operational risk than triggering occasional false alarms. The

confusion matrix is given as follows:

$$\text{Confusion Matrix} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix} = \begin{bmatrix} 196649 & 4494 \\ 1068 & 151995 \end{bmatrix}$$

The average inference latency on the Raspberry Pi 4 is 0.70 ms per sample, as computed using (1) and reported in Table 12. This corresponds to a maximum theoretical throughput of approximately 1429 samples per second on a single device. Such latency enables near real-time packet-level inference at the network edge, which is essential for timely DDoS detection in constrained environments. This cross-dataset evaluation demonstrates the model’s generalization capability under realistic hardware and quantization constraints in a low-power edge environment.

2) CICIoT2025 DATASET

The CICIoT2025 dataset is a recent IIoT traffic dataset that contains benign flows and multiple attack behaviours. The dataset provides traffic segments in separate CSV files, each containing flows captured over different time windows, such as 2, 3, 5, 8, and 10 seconds. The attack category reflects network-level malicious activity, including packet bursts and traffic patterns consistent with DDoS behaviour. In this work, the dataset is prepared in a binary format to support intrusion detection.

The process starts by combining all attack and benign CSV files into a single dataset. A set of fields that behave as explicit labels or metadata identifiers is removed to avoid leakage. This removal includes fields such as Number, label, attack_type, and related categorical indicators. The target variable is the outcome, and the feature matrix is constructed by removing the target field. Timestamp-related columns, including timestamp, are also removed because they act as implicit identifiers and degrade model generalisation.

Numeric fields are retained, infinite values are replaced with the median, missing values are filled with column-wise medians, and constant features are removed. After preprocessing, the dataset contains 238828 samples, with 129236 attack samples and 109592 benign samples. The final feature dimension is 71. Stratified sampling is applied to create training and test splits of 70% and 30%, respectively. The summary of the CICIoT2025 dataset, including total samples, attack and benign samples, is summarized in Table 11.

All features are scaled using StandardScaler, fitted on the training data, and applied to the test data. These steps establish a consistent, reproducible configuration for training the TCN model and converting it to TFLite.

Following model conversion, the INT8 TFLite model is deployed on a Raspberry Pi 4 using the proposed edge pipeline. The TCN model is evaluated on the CICIoT2025 dataset under identical settings. The deployment metrics, including accuracy and precision, recall, F1-score, ROC-AUC, and average inference time, are reported in Table 12. On the CICIoT2025 dataset, the model produces lower accuracy (e.g., 65%) due to greater feature overlap between

TABLE 12. Deployment performance of the lightweight TCN (TFLite on Raspberry Pi 4) on the CICIoT2023 and CICIoT2025 datasets.

Dataset	Acc. (%)	Prec. (%)	Rec. (%)	F1 (%)	ROC-AUC (%)	Inf. time (ms/sample)
CICIoT2023	98.43	97.13	99.30	98.20	99.55	0.70
CICIoT2025	65.80	62.99	89.26	73.86	75.35	1.29

benign and malicious flows. However, recall of 89.26% and ROC-AUC remain consistent with the overall detection objective. The inference time on the Raspberry Pi 4 increases to 1.29 ms/sample. The latency difference is independent of the number of samples in the dataset. The Raspberry Pi processes one sample at a time, and computation time depends on the size of the feature vector fed into the model. CICIoT2025 provides 71 input features per sample, while CICIoT2023 provides 38 features per sample. A larger feature vector increases the number of operations in the first convolutional layers, thereby increasing per-sample processing time. This effect explains why CICIoT2025 exhibits a slightly higher inference time of 1.29 ms than CICIoT2023 (0.70 ms). The difference remains small and stays within real-time requirements for IIoT settings.

Moreover, in the confusion matrix, the first row corresponds to benign samples, and the second row to attack samples. The model identifies a high proportion of attack samples, resulting in high recall. However, a substantial number of benign samples appear in the attack column, and this pattern reduces accuracy and precision in Table 12. This behaviour indicates the close statistical similarity between benign and attack traffic within CICIoT2025. As a result, the classification boundary becomes harder to maintain on edge hardware.

$$\text{Confusion Matrix} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix} = \begin{bmatrix} 12542 & 20336 \\ 4165 & 34606 \end{bmatrix}$$

The TCN model is evaluated from two perspectives: using a large dataset with fewer features (CICIoT2023) and a smaller dataset with a larger feature set (CICIoT2025), both on the Raspberry Pi 4. The results demonstrate that per-sample latency is related to the dimensionality of the input features rather than to the dataset size [49]. This trend also appears in our MQTT and UDP-based DDoS datasets, where the *UL-ECE-UDP-DDoS-H-IoT2025* dataset, with 13 features, exhibits a higher latency of 0.27 ms per sample.

E. SECURITY CONSIDERATIONS FOR EDGE TFLITE DEPLOYMENT

Security considerations focus on preserving the integrity, confidentiality, and authenticity of data, deployment code, and TFLite-based ML/DL models in security-critical edge deployments [50]. This work focuses on application-layer DDoS detection, in which the Raspberry Pi 4 operates as a local edge node in diverse IoT environments. In this setting, traffic-derived features are analyzed locally on the Raspberry Pi, and the DDoS detection outputs are displayed via a local monitoring interface. Deploying a TFLite model on

a resource-constrained edge device introduces risks such as misconfiguration, model tampering, privilege misuse, and resource abuse, which require explicit analysis at the deployment level [51], [52]. The potential deployment of cyber threats is as follows:

- *Model and deployment tampering:* A malicious actor modifies the TFLite model file or its deployment scripts to alter detection logic or disable the DDoS detection service.
- *Unauthorized device access:* An adversary gains unauthorized physical or remote access to the edge device (e.g., a Raspberry Pi 4), enabling the detection service to be stopped, bypassed, or reconfigured.
- *Inference service denial-of-service:* An adversary floods the inference service with excessive or malformed traffic-derived input samples, causing CPU exhaustion and increased DDoS detection latency or service interruption.
- *Model extraction and reverse engineering:* An unauthorized entity copies the TFLite model file stored on the Raspberry Pi to analyze its structure or reuse it without authorization, exposing its internal architectural and quantization design details.

Given the identified deployment-level threats, a proactive mitigation strategy is necessary to preserve the integrity and reliability of the edge inference process. This strategy focuses on protecting the deployed TFLite model, the host edge device (e.g., Raspberry Pi 4), and the inference pipeline against practical attack surfaces in resource-constrained edge environments [50]. The corresponding mitigation mechanisms are as follows:

- *Model integrity protection:* The deployment design includes integrity verification of the TFLite model and associated scripts using cryptographic file hashing, enabling detection of unauthorized modification before runtime loading.
- *Privilege minimization:* The inference service is designed to operate under restricted system privileges or a dedicated user account to limit the impact of device compromise.
- *Resource protection:* Rate-control mechanisms can be applied at the inference interface to prevent abuse by excessive request injection or CPU resource exhaustion.
- *Process isolation:* The inference process can be deployed in a lightweight container or sandbox environment to isolate it from the host system and limit lateral impact in the event of a compromised process.
- *Data locality and exposure reduction:* All inference is executed locally on the edge device, and extracted traffic

features are not transmitted outside the node, reducing exposure to remote adversaries.

The above measures address the primary practical security risks associated with deploying a TFLite-based DDoS detection model on a low-power edge platform.

F. CASE STUDIES

To highlight the practical relevance of deploying the lightweight TCN model in TFLite form on low-power edge devices, this section outlines five real-world case studies that demonstrate its integration across IoT, H-IoT, and IIoT environments.

- 1) *Home Healthcare Gateways*: Home IoT gateway devices that support precision healthcare and aggregate data from wearable sensors such as heart-rate monitors, oximeters, and motion trackers could use the model to detect abnormal spikes in network traffic [53]. These spikes (e.g., sudden bursts of MQTT publish requests) may indicate attempts by compromised devices to overwhelm the gateway. The low-energy overhead enables continuous, on-device monitoring without affecting battery-powered nodes.
- 2) *Smart Hospital Ward Monitoring*: A Raspberry Pi can operate as a local edge gateway in hospital wards, where it connects to H-IoT devices such as heart-rate sensors, temperature monitors, and wearable patient trackers [54]. In this setting, the TCN TFLite model could run directly on the edge device to detect abnormal network patterns generated by compromised devices. The measured mean inference latency (0.19–0.27 ms) enables real-time identification of DDoS-induced congestion that may disrupt patient-critical data streams.
- 3) *Remote Health Wearables and Telemedicine Kits*: Portable diagnostic and telemedicine kits (e.g., glucose meters and ECG patches) used in rural clinics, emergency response teams, or mobile medical vans often depend on weak local WiFi or temporary portable routers to transmit patient readings [55]. These unreliable connections can become congested by unsolicited network traffic, which may introduce delays during remote consultations or reduce the responsiveness of diagnostic data transfers. Deploying the TCN TFLite model on a Raspberry Pi that controls the kit provides a lightweight mechanism for tracking connection quality and maintaining timely data delivery in resource-limited medical settings.
- 4) *Industrial Sensor Clusters*: Edge controllers in IIoT environments, such as manufacturing floors and process plants, are capable of embedding the model to protect PLCs and high-frequency sensor clusters from DDoS bursts [56]. The model's ability to operate reliably across different input dimensions and communication patterns makes it suitable for IIoT

deployments where sensors often transmit heterogeneous, protocol-specific traffic. For example, the cross-dataset evaluation on the edge device, discussed in Section IV-D, demonstrates that the model remains stable even when operating on a higher-dimensional feature space (38 features).

- 5) *Smart City Environmental Nodes*: Air-quality sensors, roadside traffic detectors, and weather-monitoring units typically forward periodic measurements through lightweight edge gateways with limited compute resources [57]. These gateways are susceptible to abnormal volumes of unsolicited network traffic originating from outside the sensor network, which can saturate their communication links. Deploying the TCN TFLite model on the gateway enables early detection of traffic anomalies associated with DDoS activity, supporting uninterrupted delivery of sensor data to municipal servers.

G. SECURITY ANALYSIS AND ATTACK COVERAGE ACROSS DIVERSE IOT ENVIRONMENTS

In this paper, the lightweight TCN TFLite model is deployed to detect traffic anomalies on the Raspberry Pi 4. Its primary focus is identifying DDoS attacks, which typically manifest as rapid increases in packet rates or other abnormal communication patterns across H-IoT, IoT, and IIoT devices. Beyond DDoS, man-in-the-middle variants such as packet injection, duplication, or relaying can also introduce traffic deviations in these environments [58]. Packet injection, duplication, or relay at higher-than-normal rates alters message frequency and traffic volume, and these changes are reflected in the MQTT and UDP-based features in our datasets. Under these conditions, the TCN TFLite model can flag the anomaly because it has learned these frequency-based patterns during training.

General IoT and IIoT threats also arise from irregular communication patterns rather than increased traffic volume [59]. Replay attempts may introduce repeated message patterns, brute-force bursts may produce short, periodic authentication clusters, and malware scanning may generate probe sequences across ports or devices [60]. These pattern-level deviations differ from normal device behaviour and can be detected by the TCN TFLite model when they appear in the input features. The cross-dataset results in Table 12 demonstrate that the model continues to identify these traffic-pattern anomalies on the CICIOT2025 dataset, despite the diverse device behaviours and noisier IIoT traffic.

Other attack classes, such as firmware compromise, eavesdropping, side-channel leakage, or ransomware activation, operate at layers that do not produce distinctive changes in traffic patterns [51]. These classes are typically mitigated through cryptographic, firmware-level, or device-level protections [52]. In this setting, the model could contribute to a broader multilayer defence by covering the network-traffic dimension of the threat surface.

H. DISCUSSION OF TCN SELECTION AND EDGE DEPLOYMENT

This discussion focuses on the rationale for selecting the TCN model and its deployment on the Raspberry Pi 4. The TCN model is used in this research for its lightweight architecture and computational efficiency, as demonstrated in previous work [1], which showed that it required fewer parameters than CNNs, LSTMs, and traditional models such as decision trees and k-nearest neighbors (KNN). This advantage is further reinforced in the present research through comparison with prior models, as discussed in Section IV-A. In addition to its efficiency, the TCN model demonstrates preliminary evidence of generalization and scalability across the evaluated datasets and experimental settings. It achieves high accuracy on the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets and shows competitive performance on selected existing datasets, including the real-world IoMT dataset WUSTL-EHMS-2020 [1], [61]. These results indicate the potential applicability of TCN for handling diverse IoT and IoMT scenarios within the evaluated experimental setup. Therefore, the TCN model is converted to TFLite and deployed on a resource-constrained edge device, such as the Raspberry Pi 4.

Although the reduction in parameter count is relatively small ($63,617 \rightarrow 63,136$), this outcome is expected, as TFLite conversion focuses on optimizing model storage and computation rather than on changing the underlying architecture. The practical benefit is reflected in the substantial decrease in model size ($248.5 \text{ KB} \rightarrow 89.1 \text{ KB}$), which directly improves feasibility for deployment on edge devices with limited storage and memory resources.

The power evaluation further indicates that the additional energy overhead of deploying the TCN TFLite model is minimal on the Raspberry Pi 4, even when handling different feature dimensions (4 in the MQTT-based dataset versus 13 in the UDP-based dataset). The model maintains efficiency across MQTT and UDP-based H-IoT traffic, supporting its feasibility for deployment in IoT, H-IoT, and IIoT environments.

V. CONCLUSION

This paper adopts a previously proposed TCN model and investigates its deployment on low-power edge devices for DDoS detection in IoT, H-IoT, and IIoT environments. The model, trained and evaluated on the *UL-ECE-MQTT-DDoS-H-IoT2025* and *UL-ECE-UDP-DDoS-H-IoT2025* datasets, achieves high accuracy in identifying DDoS attacks. Furthermore, conversion to the TFLite format enables lightweight deployment on resource-constrained edge devices (e.g., Raspberry Pi 4). Through quantization, the model maintains high detection accuracy while reducing both latency and model size. The deployment of the TCN TFLite model on the Raspberry Pi 4 demonstrates low, stable power consumption during inference across both datasets, highlighting its feasibility for resource-constrained

edge environments within the scope of the study. The deployment evaluation also encompasses public IoT and IIoT datasets, enabling a broader, more representative assessment across multiple deployment domains. Future work involves deploying the $\sim 89.1 \text{ KB}$ INT8 TCN on resource-constrained IoT devices (e.g., Raspberry Pi Pico and ESP32) using TensorFlow Lite for Microcontrollers. Additionally, future work will evaluate the model on multiclass datasets and explore alternative, lightweight architectures to enhance performance and resilience against emerging cyber attacks. This research is limited to a single edge platform (Raspberry Pi 4), which may limit the direct generalizability of the deployment results to other hardware platforms.

REFERENCES

- [1] M. Akhi, C. Eising, and L. Luxmi Dhirani, "TCN-based DDoS detection and mitigation in 5G healthcare-IoT: A frequency monitoring and dynamic threshold approach," *IEEE Access*, vol. 13, pp. 12709–12733, 2025.
- [2] L. L. Dhirani, E. Armstrong, and T. Neue, "Industrial IoT, cyber threats, and standards landscape: Evaluation and roadmap," *Sensors*, vol. 21, no. 11, p. 3901, 2021.
- [3] M. A. Khatun, S. F. Memon, C. Eising, and L. L. Dhirani, "Machine learning for healthcare-IoT security: A review and risk mitigation," *IEEE Access*, vol. 11, pp. 145869–145896, 2023.
- [4] S. A. Bustin and K. A. Jellinger, "Advances in molecular medicine: Unravelling disease complexity and pioneering precision healthcare," *Int. J. Mol. Sci.*, vol. 24, no. 18, 2023, Art. no. 14168.
- [5] F. S. Costa et al., "FASTEN IIoT: An open real-time platform for vertical, horizontal and end-to-end integration," *Sensors*, vol. 20, no. 19, p. 5499, 2020.
- [6] O. T. Modupe et al., "Reviewing the transformational impact of edge computing on real-time data processing and analytics," *Comput. Sci. IT Res. J.*, vol. 5, no. 3, pp. 693–702, 2024.
- [7] "Landscape of edge computing standards," 2023. Accessed: Mar. 27, 2025. [Online]. Available: <https://tinyurl.com/5xwstsmh>
- [8] L. L. Dhirani and T. Neue, "Securing industry 5.0 using 6 σ cybernetic framework," *Mehran Univ. Res. J. Eng. Technol.*, vol. 43, pp. 220–238, Apr. 2024.
- [9] P. Lohan, B. Kantarci, M. A. Ferrag, N. Tihanyi, and Y. Shi, "From 5G to 6G networks: A survey on AI-based jamming and interference detection and mitigation," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 3920–3974, 2024.
- [10] A. A. Almuqren, "Cybersecurity threats, countermeasures and mitigation techniques on the IoT: Future research directions," *J. Cyber Security Risk Auditing*, vol. 1, no. 1, pp. 1–11, 2025.
- [11] H. Meagher and L. L. Dhirani, *Cyber-Resilience, Principles, and Practices*. Cham, Switzerland: Springer Nat., 2024, pp. 57–74. [Online]. Available: https://doi.org/10.1007/978-3-031-45162-1_4
- [12] E. Gyamfi and A. Jurcut, "Intrusion detection in Internet of Things systems: A review on design approaches leveraging multi-access edge computing, machine learning, and datasets," *Sensors*, vol. 22, no. 10, p. 3744, 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/10/3744>
- [13] "IDS VS SIEM: Essential guide to OT security," 2025. Accessed: Mar. 27, 2025. [Online]. Available: <https://www.otorio.com/blog/intrusion-detection-systems-ids/>
- [14] L. He, L. Fan, X. Lei, P. Fan, A. Nallanathan, and G. K. Karagiannidis, "The road toward general edge intelligence: Standing on the shoulders of foundation models," *IEEE Commun. Mag.*, vol. 63, no. 9, pp. 164–170, Sep. 2025.
- [15] G. Akar, S. Sahmoud, M. Onat, Ü. Cavusoglu, and E. Malondo, "L2D2: A novel LSTM model for multi-class intrusion detection systems in the era of IoMT," *IEEE Access*, vol. 13, pp. 7002–7013, 2025.

- [16] S. Z. Omer, F. Hashim, A. Sali, and F. A. Ahmad, "Binary classification of low-rate DoS attacks using long short-term memory feed-forward (LSTM-FF) intrusion detection system (IDS)," *Eng. Sci. Technol. Int. J.*, vol. 66, Jun. 2025, Art. no. 102049.
- [17] M. V. Assis, L. F. Carvalho, J. Lloret, and M. L. Proença Jr, "A GRU deep learning system against attacks in software defined networks," *J. Netw. Comput. Appl.*, vol. 177, Mar. 2021, Art. no. 102942.
- [18] M. A. Khatun, M. Bhattacharya, C. Eising, and L. L. Dhirani, "Time series anomaly detection with CNN for environmental sensors in healthcare-IoT," 2024, *arXiv:2407.20695*.
- [19] F. M. Aswad, A. M. S. Ahmed, N. A. M. Alhammedi, B. A. Khalaf, and S. A. Mostafa, "Deep learning in distributed denial-of-service attacks detection method for Internet of Things networks," *J. Intell. Syst.*, vol. 32, no. 1, 2023, Art. no. 20220155.
- [20] M. Nawaz, S. Tahira, D. Shah, S. Ali, and M. Tahir, "Lightweight machine learning framework for efficient DDoS attack detection in IoT networks," *Sci. Rep.*, vol. 15, no. 1, 2025, Art. no. 24961.
- [21] X. Wang and W. Jia, "Optimizing edge AI: A comprehensive survey on data, model, and system strategies," 2025, *arXiv:2501.03265*.
- [22] Z. Lv et al., "Efficient deployment of peanut leaf disease detection models on edge AI devices," *Agriculture*, vol. 15, no. 3, p. 332, 2025.
- [23] J. Wei, M. Fan, X. Zhang, W. Jiao, H. Wang, and T. Liu, "Stealthy backdoor attack to real-world models in android apps," 2025, *arXiv:2501.01263*.
- [24] M. Akhi, C. Eising, and L. L. Dhirani, "Datasets for distributed denial-of-service detection in healthcare Internet of Things environments," *Data Brief*, vol. 63, Dec. 2025, Art. no. 112222. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352340925009436>
- [25] M. Akhi, C. Eising, and L. L. Dhirani, 2025, "Mirzaakhi/UL-ECE-DDoS-H-IoT-Datasets2025: Initial release of UL-ECE-DDoS-H-IoT2025 datasets (v1.0.0)," Dataset. [Online]. Available: <https://data.mendeley.com/datasets/2bw34ght8b/2>
- [26] N. T. Newaz and E. Hanada, "An approach to fall detection using statistical distributions of thermal signatures obtained by a stand-alone low-resolution IR array sensor device," *Sensors*, vol. 25, no. 2, p. 504, 2025.
- [27] V. Krasteva, T. Stoyanov, and I. Jekova, "Implementing deep neural networks on ARM-based microcontrollers: Application for ventricular fibrillation detection," *Appl. Sci.*, vol. 15, no. 4, p. 1965, 2025.
- [28] O. Jouini, K. Sethom, A. Namoun, N. Aljohani, M. H. Alanazi, and M. N. Alanazi, "A survey of machine learning in edge computing: Techniques, frameworks, applications, issues, and research directions," *Technologies*, vol. 12, no. 6, p. 81, 2024. [Online]. Available: <https://www.mdpi.com/2227-7080/12/6/81>
- [29] "How to check power consumption on raspberry Pi," Dec. 2023. Accessed: Sep. 2, 2025. [Online]. Available: <https://linuxconfig.org/how-to-check-power-consumption-on-raspberry-pi>
- [30] G. Bekaroo and A. Santokhee, "Power consumption of the raspberry Pi: A comparative analysis," in *Proc. IEEE Int. Conf. Emerg. Technol. Innovative Bus. Practices Transf. Soc. (EmergiTech)*, 2016, pp. 361–366.
- [31] H. C. Altunay and Z. Albayrak, "A hybrid CNN+ LSTM-based intrusion detection system for industrial IoT networks," *Eng. Sci. Technol. Int. J.*, vol. 38, Feb. 2023, Art. no. 101322.
- [32] R. B. Said, Z. Sabir, and I. Askerzade, "CNN-BiLSTM: A hybrid deep learning approach for network intrusion detection system in software-defined networking with hybrid feature selection," *IEEE Access*, vol. 11, pp. 138732–138747, 2023.
- [33] N. B. Singh, M. M. Singh, A. Sarkar, and J. K. Mandal, "A novel wide & deep transfer learning stacked GRU framework for network intrusion detection," *J. Inf. Security Appl.*, vol. 61, Sep. 2021, Art. no. 102899.
- [34] C. Liu, Y. Liu, Y. Yan, and J. Wang, "An intrusion detection model with hierarchical attention mechanism," *IEEE Access*, vol. 8, pp. 67542–67554, 2020.
- [35] M. Hassan Zaib, "NSL-KDD dataset (CSV version)," Accessed: Aug. 22, 2025. [Online]. Available: <https://www.kaggle.com/datasets/hassan06/nslkdd>
- [36] M. Akhi, C. Eising, and L. L. Dhirani, "WSN-DDoS-attack-H-IoT2023 dataset for time series anomaly detection in healthcare-IoT," 2025. [Online]. Available: <https://zenodo.org/records/15701881>
- [37] N. Dash, S. Chakravarty, A. K. Rath, N. C. Giri, K. M. AboRas, and N. Gowtham, "An optimized LSTM-based deep learning model for anomaly network intrusion detection," *Sci. Rep.*, vol. 15, no. 1, p. 1554, 2025.
- [38] D. Mrozek, "A review of cloud computing technologies for comprehensive microRNA analyses," *Comput. Biol. Chem.*, vol. 88, Oct. 2020, Art. no. 107365.
- [39] C. Tsallis, P. Papageorgas, D. Piromalis, and R. A. Munteanu, "Application-wise review of machine learning-based predictive maintenance: Trends, challenges, and future directions," *Appl. Sci.*, vol. 15, no. 9, p. 4898, 2025.
- [40] N. Faruqi et al., "SafetyMed: A novel IoMT intrusion detection system using CNN-LSTM hybridization," *Electronics*, vol. 12, no. 17, p. 3541, 2023.
- [41] E. Manor and S. Greenberg, "Custom hardware inference accelerator for tensorflow lite for microcontrollers," *IEEE Access*, vol. 10, pp. 73484–73493, 2022.
- [42] K. Liu et al., "Low-bit model quantization for deep neural networks: A survey," 2025, *arXiv:2505.05530*.
- [43] Q. Zhou et al., "Octo:[INT8] training with loss-aware compensation and backward quantization for tiny on-device learning," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2021, pp. 177–191.
- [44] TensorFlow Model Optimization Team, "Quantization aware training," 2023. Accessed: Aug. 31, 2025. [Online]. Available: https://www.tensorflow.org/model_optimization/guide/quantization/training
- [45] C. R. Banbury et al., "Benchmarking TinyML systems: Challenges and direction," 2020, *arXiv:2003.04821*.
- [46] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*. New York, NY, USA: McGraw-Hill, 2011.
- [47] A. Abdulhameed and Q. Memon, "An improved trapezoidal rule for numerical integration," in *Proc. J. Phys. Conf. Ser.*, 2021, Art. no. 12104.
- [48] M. Akhi, "UL-ECE-IoT DDoS detection power consumption recordings on raspberry Pi 4," 2025. [Online]. Available: <https://github.com/mirzaakhi/UL-ECE-IoT-DDoS-Detection-Power-Consumption-RaspberryPi4>
- [49] U. Thapa, B. M. Pati, A. Taparugssanagorn, and L. Mucchi, "Generalizable hybrid wavelet-deep learning architecture for robust arrhythmia detection in wearable ECG monitoring," *Sensors*, vol. 25, no. 21, p. 6590, 2025.
- [50] V. Padmavathi and R. Saminathan, "Security for the internet of things," in *Computer and Information Security Handbook*. Amsterdam, The Netherlands: Elsevier, 2025, pp. 353–368.
- [51] V. Jeyalakshmi, V. Balan, and V. Benitha, "Security issues in IoT: Perspective review," *Comput. Netw. Commun.*, vol. 3, no. 1, pp. 101–122, 2025.
- [52] H. Huang and J. Wang, "Systematic literature review on security mechanisms for IoT device firmware update," 2025. [Online]. Available: <http://dx.doi.org/10.2139/ssrn.5359954>
- [53] M. Ianculescu et al., "Enhancing connected health ecosystems through IoT-enabled monitoring technologies: A case study of the moni4healthy system," *Sensors*, vol. 25, no. 7, p. 2292, 2025.
- [54] F. Serepas, I. Papias, K. Christakis, N. Dimitropoulos, and V. Marinakis, "Lightweight embedded IoT gateway for smart homes based on an ESP32 microcontroller," *Computers*, vol. 14, no. 9, p. 391, 2025.
- [55] P. Kumar et al., "Evaluation of diagnostic services in rural and remote areas: Bottlenecks, success stories, and solutions," *J. Surg. Specialties Rural Pract.*, vol. 6, no. 1, pp. 32–37, 2025.
- [56] S. Latif, D. Djenouri, Z. Idrees, J. Ahmad, Q.-U.-A. Mastoi, and Z. Zou, "Hardware security modules for secure communications in the Industrial Internet of Things," *IEEE Commun. Surv. Tut.*, early access, Aug. 18, 2025, doi: [10.1109/COMST.2025.3600161](https://doi.org/10.1109/COMST.2025.3600161).
- [57] A. Pozzebon, A. Fort, B. Tellini, and A. Buffi, "Real-time sensing tools for the detection of sudden extreme weather events," in *Proc. ICSI*, 2025, pp. 561–723.
- [58] F. Aliyu, U. Baroudi, and M. Younis, "Enhancing base station anonymity in WSNs: A security framework using ring routing, fake packet injection, and dynamic coverage," *Clust. Comput.*, vol. 28, no. 16, p. 1020, 2025.
- [59] C. Sheng et al., "Network traffic fingerprinting for IIoT device identification: A survey," *IEEE Trans. Ind. Informat.*, vol. 21, no. 5, pp. 3541–3554, May 2025.

- [60] R. Flood, "Enhancing trust in NIDS: From flawed benchmarks to formal guarantees," Ph.D. dissertation, School Inf., Univ. Edinburgh, Edinburgh, U.K., 2025.
- [61] R. Jain and K. Trivedi, 2020, "Electroencephalogram (EEG) health monitoring system dataset," Accessed: Dec. 14, 2024. [Online]. Available: <https://www.cse.wustl.edu/~jain/ehms/index.html>



CIARÁN EISING (Senior Member, IEEE) received the B.E. and Ph.D. degrees in electronic and computer engineering from NUI Galway in 2003 and 2010, respectively. From 2009 to 2020, he was a Computer Vision Architect and a Senior Expert with Valeo. He is a Professor of AI with the University of Limerick.



MIRZA AKHI (Member, IEEE) received the B.Sc. degree in computer science and engineering and the M.Sc. degree in computer science and engineering from Jagannath University, Bangladesh, focusing on Internet of Things (IoT) security and machine learning in 2019. She is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, University of Limerick, Ireland. Her ongoing research delves into healthcare-IoT security and utilizing AI/ML innovations. She has an exemplary academic background, underscored by a scholarship from the Science Foundation Ireland Centre for Research Training in Foundations of Data Science in 2022. During the master's program, she secured the first position in her class. She has published research papers in renowned IEEE international conferences and reputable journals, reflecting her commitment to the professional community. Her research interests include healthcare-IoT, artificial intelligence, machine learning, data analysis, network security, and systems architecture.



LUBNA LUXMI DHIRANI (Senior Member, IEEE) is an Associate Professor with the Department of Electronic and Computer Engineering, University of Limerick (UL). She is the Course Director of the B.Sc. Cybersecurity Practitioner Apprenticeship Program taught at UL. She is the PI from UL for the EU-funded CATCH_VR Project. Her research interests include cybersecurity, cloud, the IoT, IT/OT, standards, cyber law and regulations, and privacy and risk management in emerging technologies. She has delivered several technical workshops, tutorials, and masterclasses. She actively participates in initiatives supporting women in STEM, is a former DELL StemAspire mentee, and was honored with the SFI Confirm EPE Champions Award in 2021. She is a Lero-SFI Research Centre for Software Member and the first WIE Ambassador from Ireland in the IEEE WIE (UK&I section).